WEJ LAB

Regularization in Regression

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-21

RUTGERS HEALTH
Center for Data Science

Bias-Variance tradeoff

- **Bias–variance tradeoff:** we can reduce the variance of estimates by accepting some bias
- **Bias–variance dilemma:** we can't minimize both at once, and both limit how well a model generalizes beyond its training set

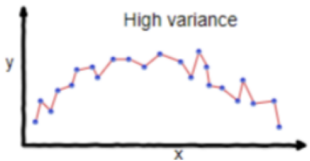
(Source: Wikipedia)

Bias-Variance tradeoff

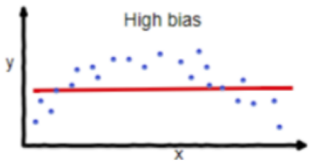
- **Bias:** error from wrong assumptions; misses real relationships (**underfitting**)
- **Variance:** error from sensitivity to the training set; models the noise (**overfitting**)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

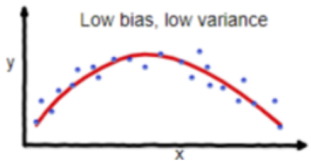
Bias-Variance tradeoff



overfitting

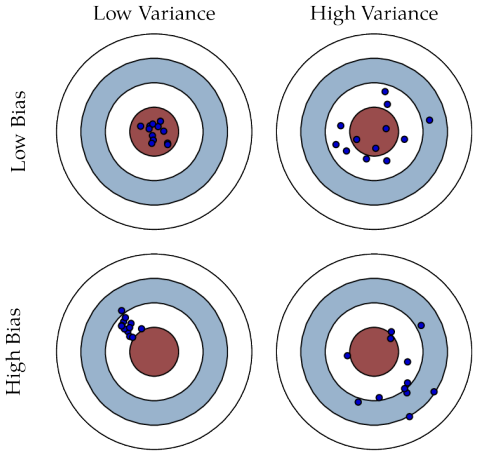


underfitting



Good balance

Bias-Variance tradeoff



Bias-Variance tradeoff

A central problem in supervised learning. We want a model that:

- captures the patterns in the training data, **and**
- generalizes to new data

Usually we can't have both:

- High variance: fits training data well, but overfits noise
- High bias: simpler, but underfits real patterns

(Source: Wikipedia)

Regularization in Machine Learning

- With small samples or noisy data, coefficient estimates can be inaccurate (overfitting)
- **Regularization** restricts or shrinks estimates toward zero:
  - reduces the impact of unimportant features
  - adds a little bias to cut variance and overfitting
  - improves performance on new data[1]

[1] Adapted from: https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning

Regularization in Machine Learning

Examples:

- Regression: penalize coefficients for parsimonious models (variable selection)
- K-means: restrict clusters to avoid redundant groups
- Neural networks: constrain the weights
- Random forests: limit tree depth

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Ridge regression

Ridge regression **shrinks** coefficients by penalizing their size:

$$\hat{\beta}^{ridge} = \inf_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\},$$

where $\lambda \geq 0$ controls the shrinkage: larger λ, more shrinkage toward 0.

Ridge regression

In matrix form:

$$\hat{\beta}^{ridge} = \inf_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \right\}.$$

with closed-form solution:

$$\hat{\beta}^{ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^T \mathbf{y}$$

Ridge regression

The ridge penalty $\lambda\beta^T\beta$ is an **L2 penalty** (squared size of the coefficients). SVMs use it too.

- Handles multicollinearity: shrinks coefficients but keeps all variables
- Does **not** select variables. For that, we use an **L1 penalty**.

Ridge regression: a Bayesian perspective

The ridge penalty is equivalent to a Normal prior on β with mean zero; λ is inversely related to the prior variance.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Lasso regression

Lasso (Least Absolute Shrinkage and Selection Operator) shrinks coefficients with an **L1** penalty:

$$\hat{\beta}^{lasso} = \inf_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \alpha \sum_{j=1}^{p} |\beta_j| \right\},$$

where $\alpha \geq 0$ controls the shrinkage: larger α, more shrinkage toward 0.

Lasso regression

- Same as ridge, with L2 $\sum_{j=1}^{p} \beta_j^2$ replaced by L1 $\sum_{j=1}^{p} |\beta_j|$
- No closed-form solution (nonlinear in the y_i)
- A large enough α sets some coefficients **exactly to zero**: the lasso does **variable selection**

Lasso vs Ridge regression

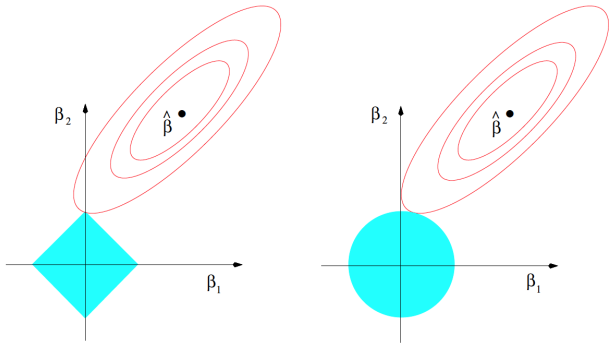


FIGURE 3.11. *Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions* $|\beta_1| + |\beta_2| \leq t$ *and* $\beta_1^2 + \beta_2^2 \leq t^2$*, respectively, while the red ellipses are the contours of the least squares error function.*

(Elements of Statistical Learning, Hastie, Tibshirani, Friedman, by Springer)

Lasso vs Ridge regression

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

(https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning)

Elastic net regularization

Ridge or lasso? Why choose! The **elastic net** minimizes:

$$\hat{\beta}^{\text{elastic net}} = \inf_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \alpha \sum_{j=1}^{p} |\beta_j| + \lambda \sum_{j=1}^{p} \beta_j^2 \right\},$$

for $\alpha \geq 0$, $\lambda \geq 0$.

- The quadratic term makes the loss strongly convex (unique minimum)
- Special cases: ridge ($\alpha = 0$), lasso ($\lambda = 0$), OLS (both 0)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Regression Regularization in R: glmnet

Regularization in R with **glmnet**:

```
install.packages("glmnet")
```

Regression Regularization in R: glmnet

For least squares (the default), glmnet minimizes:

$$\hat{\beta}^{elastic\ \ net} = \inf_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \left(\alpha \sum_{j=1}^{p} |\beta_j| + (1-\alpha) \sum_{j=1}^{p} \beta_j^2 \right) \right\},$$

with $\alpha = 1$ by default (lasso!).

Regression Regularization in R: glmnet

The package's quick start example:

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.0
```

```
data(QuickStartExample)
x <- QuickStartExample$x
y <- QuickStartExample$y
```

Regression Regularization in R: glmnet

The most basic call:

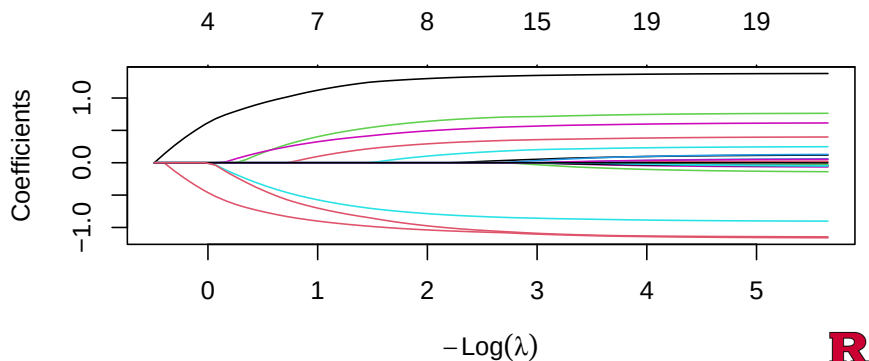
```
fit <- glmnet(x, y)
```

`fit` holds the full fitted model. Use `plot`, `print`, `coef`, and `predict` rather than extracting components directly.

Regression Regularization in R: glmnet

Coefficient paths:

```
plot(fit)
```



Regression Regularization in R: glmnet

`print` summarizes the path at each step:

```
print(fit)
```

```
##
## Call:  glmnet(x = x, y = y)
##
##    Df  %Dev  Lambda
## 1   0  0.00 1.63100
## 2   2  5.53 1.48600
## 3   2 14.59 1.35400
## 4   2 22.11 1.23400
## 5   2 28.36 1.12400
## 6   2 33.54 1.02400
## 7   4 39.04 0.93320
## 8   5 45.60 0.85030
## 9   5 51.54 0.77470
```

Regression Regularization in R: glmnet

Coefficients at a given λ:

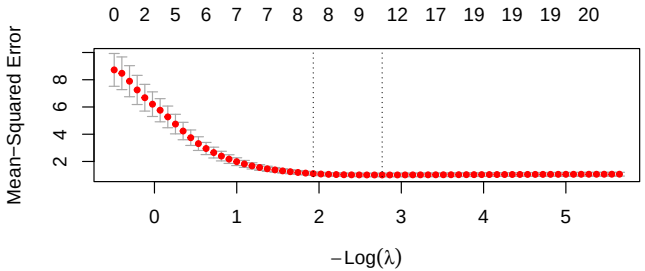
```
coef(fit, s = 0.1)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                      s=0.1
## (Intercept)  0.150928072
## V1           1.320597195
## V2           .
## V3           0.675110234
## V4           .
## V5          -0.817411518
## V6           0.521436671
## V7           0.004829335
## V8           0.319415917
```

Regression Regularization in R: glmnet

`glmnet` returns a whole sequence of models. Choose one by **cross-validation** with `cv.glmnet`:

```
cvfit <- cv.glmnet(x, y)
plot(cvfit)
```



Regression Regularization in R: glmnet

λ_{min} and its coefficients:

```
cvfit$lambda.min
```

```
## [1] 0.06284188
```

```
coef(cvfit, s = "lambda.min")
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                lambda.min
## (Intercept)  0.145832036
## V1           1.340981414
## V2           .
## V3           0.708347140
## V4           .
## V5          -0.848087765
## V6           0.554823782
## V7           0.038519738
## V8           0.347221319
## V9           .
## V10          0.010034050
## V11          0.186365265
## V12          .
## V13          .
## V14         -1.086006902
## V15         -0.004444319
## V16          .
```

Example: Logistic Regression Elastic Net

For a binary response $y \in \{0, 1\}$, logistic regression models

$$P(y = 1|\mathbf{X}) = \frac{e^{\mathbf{X}\beta}}{1 + e^{X\beta}},$$

or equivalently

$$\log \frac{P(y = 1|\mathbf{X})}{P(y = 0|\mathbf{X})} = \mathbf{X}\beta,$$

the logistic (log-odds) transformation.

Example: Logistic Regression Elastic Net

Minimize:

$$\inf_{\beta} \left\{ -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \mathbf{x}_i' \beta - \log\left(1 + e^{\mathbf{x}_i' \beta}\right) \right] + \lambda \left(\alpha \sum_{j=1}^{p} |\beta_j| + (1-\alpha) \sum_{j=1}^{p} \beta_j^2 \right) \right\},$$

the logistic regression loss plus the elastic net penalty.

- Logistic regression breaks down when $p > N$ and is unstable when $N \approx p$
- The elastic net fixes this by regularizing and selecting variables

Example: Logistic Regression Elastic Net

The glmnet binomial example:

```
library(glmnet)
data(BinomialExample)
x <- BinomialExample$x
y <- BinomialExample$y
```

Example: Logistic Regression Elastic Net

Use `family = "binomial"`:

```
fit <- glmnet(x, y, family = "binomial")
```

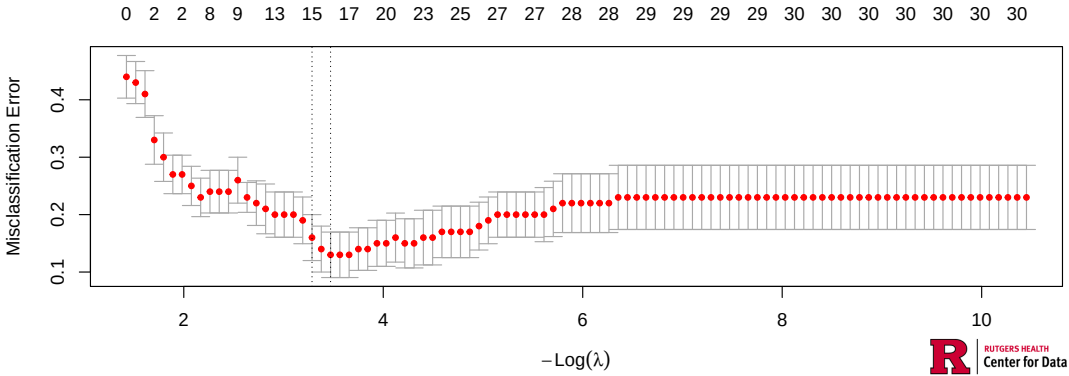
10-fold CV with misclassification error:

```
cvfit <- cv.glmnet(x, y, family = "binomial",
                   type.measure = "class")
```

Example: Logistic Regression Elastic Net

CV results and the best λ_{min}:

```
plot(cvfit)
```



Example: Logistic Regression Elastic Net

```
coef(cvfit, s = "lambda.min")
```

```
## 31 x 1 sparse Matrix of class "dgCMatrix"
##                lambda.min
## (Intercept)  0.223538008
## V1           .
## V2           0.406310353
## V3          -0.317908883
## V4          -0.848918324
## V5          -0.093799966
## V6          -0.621172683
## V7           .
## V8          -0.340441065
## V9           0.439165956
## V10         -0.979472074
## V11         -0.003318893
## V12          .
## V13          .
## V14          .
## V15          .
## V16          0.082611451
## V17          .
## V18          .
## V19          .
## V20          .
## V21          .
## V22          0.148877740
## V23          0.196265351
```

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Denver
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] glmnet_5.0   Matrix_1.7-5
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.39     codetools_0.2-20  fastmap_1.2.0     shape_1.4.6.1
##  [5] xfun_0.62         lattice_0.22-9    splines_4.5.2     iterators_1.0.14
##  [9] knitr_1.52        htmltools_0.5.9   rmarkdown_2.32    tinytex_0.61
## [13] cli_3.6.6         foreach_1.5.2     grid_4.5.2        compiler_4.5.2
## [17] rstudioapi_0.19.0 tools_4.5.2       evaluate_1.0.5    survival_3.8-6
```